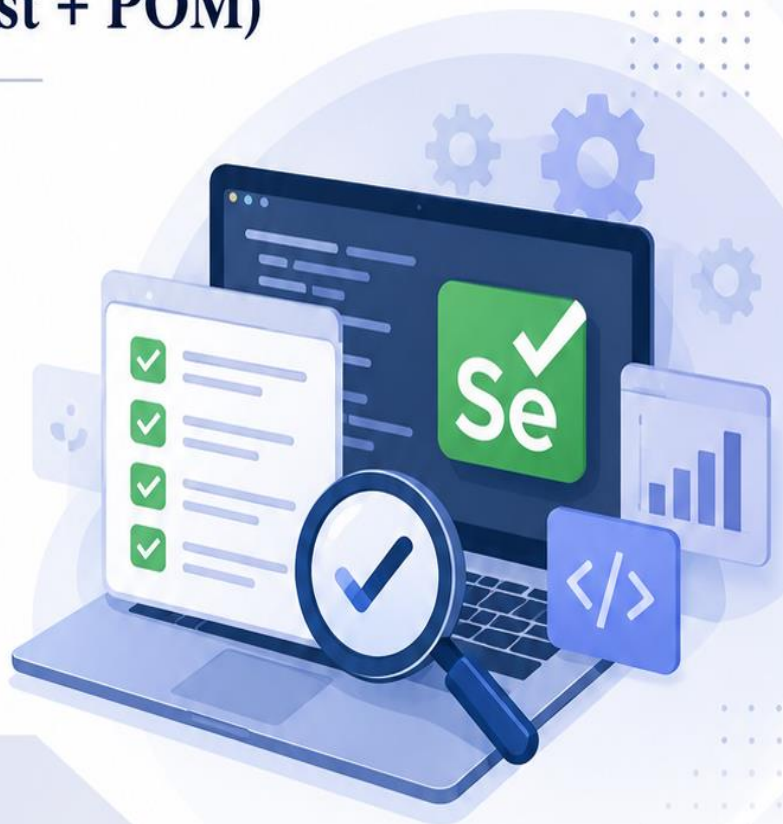


CAPSTONE ASSIGNMENT 2

Selenium Python Framework Development (Unittest + PyTest + POM)

-  Unittest
-  PyTest
-  Page Object Model (POM)
-  Utility Classes
-  Configuration Management
-  Test Data Handling (CSV)
-  Screenshots on Failure
-  HTML Reporting



NAME
Adrija Mukherjee



ENROLLMENT NO.
12023052002001



DEPARTMENT
Computer Science and Engineering



1. Objective

This project designs and implements a scalable Selenium Python automation framework that combines both Unittest and PyTest test runners under a single Page Object Model (POM) architecture. The framework automates the Login, Signup/Account-Lifecycle, and Product Search functionality of a live e-commerce practice application, and demonstrates configuration management, CSV-driven test data, screenshot capture on failure, and self-contained HTML reporting. The assignment brief called for the following capabilities, all of which are present in the delivered framework and are backed by execution evidence captured directly from the developer's own machine (see the figures throughout this report):

- A Selenium Python Automation Framework built with Unittest
- The same functional coverage re-implemented with PyTest
- A Page Object Model (POM) layer separating locators/actions from test logic
- Reusable Utility Classes (WebDriver factory, screenshot capture, CSV reader)
- Centralised Configuration Management (base URL, browser, timeouts, headless mode)
- Test Data Handling via CSV files, consumed by both runners
- Automatic Screenshot capture on test failure
- Self-contained HTML Reporting via pytest-html

2. Target Application Under Test

Rather than pointing Selenium at a hand-built demo page, the framework targets a live, publicly hosted QA practice site — automationexercise.com — described on its own homepage as a “Full-Fledged practice website for Automation Engineers”, built specifically for automation and API testing practice at any skill level.

This is a meaningful design decision: a real third-party site has genuine dynamic behaviour — ad overlays, modals, session cookies, server-rendered validation messages — which is exactly the sort of noise a production QA framework has to tolerate. The three flows chosen for automation map directly onto the assignment brief:

Flow	Live URL	What is automated
Login	/login	Valid login, invalid credentials, CSV-driven negative cases, logout
Signup / Account Lifecycle	/login → /signup → /delete_account	Full registration form, forced login state, re-login, account deletion
Product Search	/products	Keyword search, empty-result handling, add-to-cart, cart-content verification

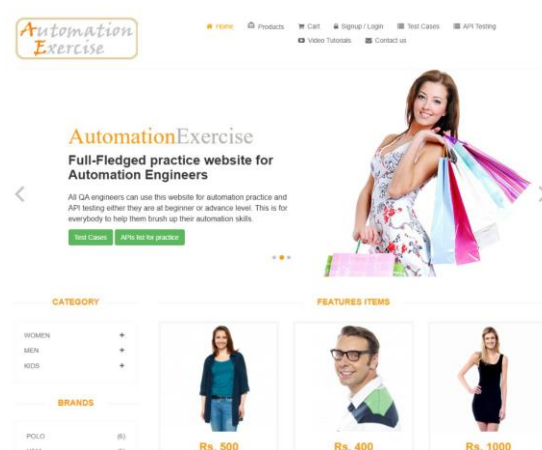


Figure 1: Home page of automationexercise.com, the live site used as the automation target.

3. Framework Architecture

The framework follows a strict separation of concerns so that test intent (what a scenario checks) never mixes with implementation detail (how an element is located or a driver is configured). Six layers work together, as shown below alongside the actual project directory structure:

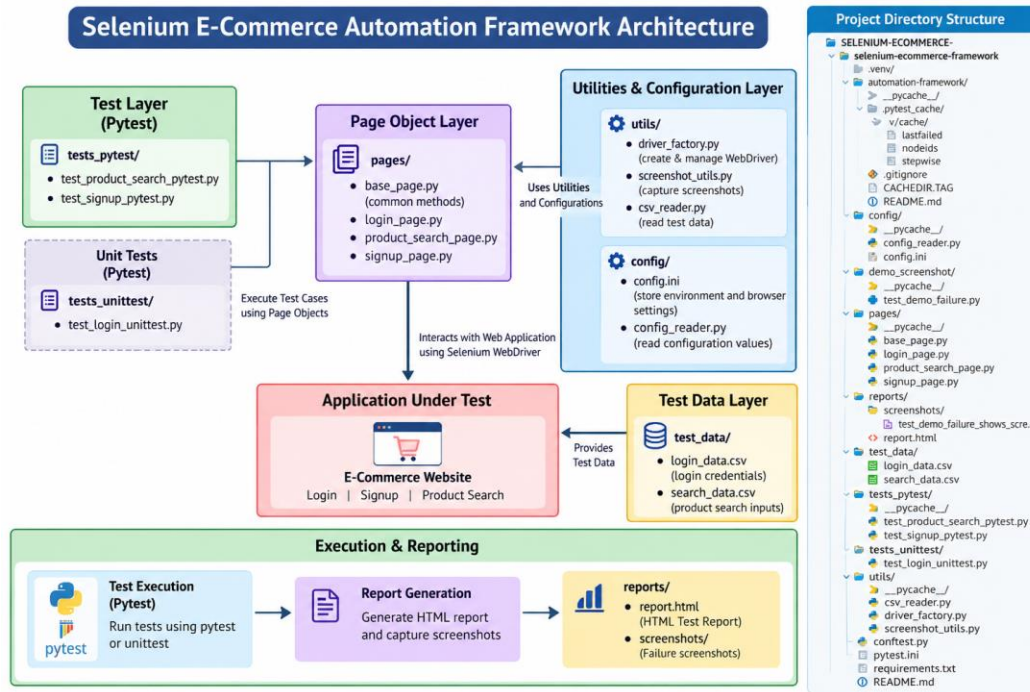


Figure 2: Framework architecture — Test Layer, Page Object Layer, Utilities & Configuration Layer, Application Under Test, Test Data Layer, and Execution & Reporting — with the corresponding project directory structure.

3.1 Layered Responsibility

Layer	Responsibility	Key file(s)
Configuration Management	Single source of truth for base URL, browser, headless flag, and all timeouts; overridable per-key via environment variables without touching code	<code>config/config.ini</code> , <code>config_reader.py</code>
Page Object Model	Encapsulates locators and page-specific actions; test files never call raw Selenium <code>find_element</code> methods	<code>pages/base_page.py</code> and per-page classes
Utility Classes	Cross-cutting concerns used by every test: driver creation, screenshot capture, CSV parsing	<code>utils/*.py</code>
Test Data	Login and search inputs kept out of test code so new cases are added by editing a spreadsheet, not Python	<code>test_data/*.csv</code>
Test Runners	Two independent suites (Unittest and PyTest) exercising the same Page Objects, proving the POM layer is runner-agnostic	<code>tests_unittest/</code> , <code>tests_pytest/</code>
Reporting & Diagnostics	Failure screenshots plus a self-contained HTML report for every PyTest run	<code>confest.py</code> , <code>pytest.ini</code>

4. Configuration Management

`config/config_reader.py` wraps Python's built-in `configparser` and adds a rule that any setting can be overridden by an environment variable of the same name, in upper case. This means the exact same code base can run locally, against staging, headless in CI, or headed on a developer's machine, without a single code change:

```

# default run
pytest tests_pytest/

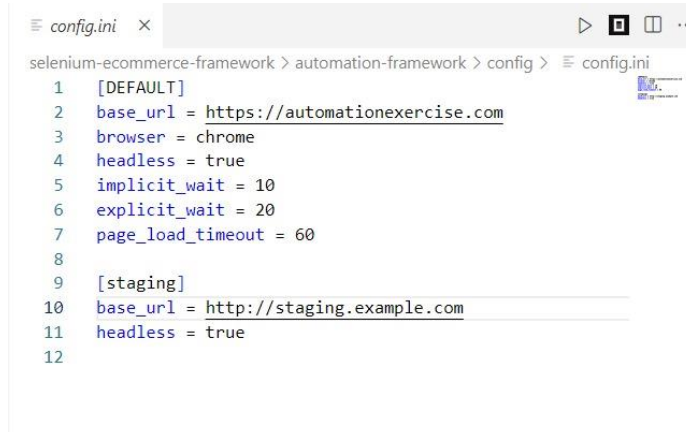
# watch it happen in a visible browser, slowed down
HEADLESS=false SLOWMO=1 pytest tests_pytest/ -v

# target the staging profile defined in config.ini

```

```
TEST_ENV=staging pytest tests_pytest/
```

Configured values include: `base_url`, `browser`, `headless`, `implicit_wait`, `explicit_wait`, `page_load_timeout`, `slowmo` (extra pause after each action, useful for demo recordings) and `keep_open_seconds` (keeps the browser open briefly after a test finishes so the final page can be inspected). Below is the actual `config.ini` used for every run in this report:



```

1 [DEFAULT]
2 base_url = https://automationexercise.com
3 browser = chrome
4 headless = true
5 implicit_wait = 10
6 explicit_wait = 20
7 page_load_timeout = 60
8
9 [staging]
10 base_url = http://staging.example.com
11 headless = true
12

```

Figure 3: *config/config.ini as used for the runs captured in this report.*

5. Page Object Model

5.1 BasePage

Every page class inherits from `BasePage`, which centralises explicit waits (`WebDriverWait` combined with `expected_conditions`) and the small set of interactions every page needs: `find`, `find_clickable`, `find_all`, `is_visible`, `click`, `type_text`, `get_text`. This is what keeps individual page classes readable — they read almost like plain English.

5.2 LoginPage

Wraps the `/login` screen. Locators target the site's own `data-qa` attributes and stable CSS selectors (`input[data-qa='login-email']`, `a[href='/logout']`), which are far less brittle than auto-generated IDs. Exposes `load()`, `login(email, password)`, `logout()`, `get_error_message()` and `is_login_successful()`. The actual implementation, as opened in the developer's editor:

```

login_page.py
selenium-e-commerce-framework > automation-framework > pages > login_page.py > LoginPage > login

1  from selenium.webdriver.common.by import By
2
3  from pages.base_page import BasePage
4  from config.config_reader import ConfigReader
5
6
7  class LoginPage(BasePage):
8      EMAIL_INPUT = (By.CSS_SELECTOR, "input[data-qa='login-email']")
9      PASSWORD_INPUT = (By.CSS_SELECTOR, "input[data-qa='login-password']")
10     LOGIN_BUTTON = (By.CSS_SELECTOR, "button[data-qa='login-button']")
11     # Rendered only after a failed login POST ("Your email or password is incorrect")
12     ERROR_MESSAGE = (By.CSS_SELECTOR, ".login-form p")
13     LOGOUT_LINK = (By.CSS_SELECTOR, "a[href='/logout']")
14     LOGGED_IN_AS = (By.PARTIAL_LINK_TEXT, "Logged in as")
15
16     def load(self):
17         self.open(ConfigReader.base_url() + "/login")
18         return self
19
20     def login(self, email, password):
21         self.type_text(self.EMAIL_INPUT, email)
22         self.type_text(self.PASSWORD_INPUT, password)
23         self.click(self.LOGIN_BUTTON)
24         return self
25
26     def logout(self):
27         self.click(self.LOGOUT_LINK)
28         return self
29
30     def get_error_message(self):
31         if self.is_visible(self.ERROR_MESSAGE, timeout=10):
32             return self.get_text(self.ERROR_MESSAGE)
33         return None
34
35     def is_login_successful(self):
36         return self.is_visible(self.LOGOUT_LINK, timeout=10)
37

```

Figure 4: pages/login_page.py — the actual LoginPage implementation.

5.3 SignupPage

Wraps the two-step registration flow (/login → /signup → confirmation). Handles the account-information form, including <select> dropdowns via Selenium's Select helper, and exposes a single convenience method, register_unique_user(), that runs the entire flow and returns the generated credentials — plus delete_account() so tests clean up after themselves and never accumulate throwaway users on the live site.

5.4 ProductSearchPage

Wraps /products: search(term), get_product_names(), get_product_card_count(), has_no_results_message(), add_first_result_to_cart() and cart_contains(name). The add-to-cart method is worth calling out specifically in a report or viva, because it demonstrates handling a real-world Selenium obstacle:

Ad overlays on the live site can intercept a native .click(), throwing ElementClickInterceptedException. The framework works around this with a JavaScript-dispatched click (driver.execute_script("arguments[0].click();", element)) after scrolling the element into view — a pattern worth explaining in a viva as evidence of debugging a real flaky-element problem, not just following a tutorial.

6. Utility Classes

Utility	Purpose
driver_factory.py	Single place that builds a configured Chrome or Firefox WebDriver (headless flag, window size, implicit wait, page-load timeout) so no test file ever constructs a driver by hand.
screenshot_utils.py	take_screenshot(driver, test_name) saves a timestamped PNG to reports/screenshots/, used identically by both the Unittest tearDown and the PyTest failure hook.
csv_reader.py	read_csv(relative_path) turns a CSV file into a list of dictionaries, the common data shape consumed by pytest.mark.parametrize and unittest's subTest loop alike.

7. Test Data Handling (CSV)

Test inputs live entirely outside the Python code, in two CSV files under `test_data/`. Adding a new negative-login case or a new search term means editing a spreadsheet — no code review of test logic required.

7.1 login_data.csv

email	password	expected_result	description
nobody@example.com	Test@1234	failure	Unregistered email
nobody@example.com	(empty)	failure	Empty password

7.2 search_data.csv

search_term	expect_results	description
jeans	true	Matches jeans products
zzz-nonexistent	false	No product should match

Both files are read once at module import time and fed into `pytest.mark.parametrize` (giving each CSV row its own named test-case ID in the PyTest output — see Figure 7) and into a `unittest.subTest` loop inside the Unittest suite (giving each row its own pass/fail line without needing a separate test method per row).

8. Test Suite Coverage

8.1 Unittest Suite — `tests_unittest/test_login_unittest.py`

Uses `setUpClass/tearDownClass` to register one throwaway account for the whole class (via `SignupPage`) and delete it afterwards, and per-test `setUp/tearDown` for driver lifecycle. `tearDown` inspects the test outcome and calls `take_screenshot()` only when the test actually failed.

Test method	What it verifies
<code>test_valid_login_shows_shop_section</code>	A correctly registered user can log in and reach an authenticated state (logout link visible)
<code>test_invalid_login_shows_error</code>	An incorrect password produces the site's inline error message
<code>test_login_data_driven_from_csv</code>	Every row of <code>login_data.csv</code> is exercised inside one test via <code>subTest</code> , each reported individually

8.2 PyTest Suite

`tests_pytest/test_signup_pytest.py` drives the complete account lifecycle: register → verify logged-in → log out → log back in with the new credentials → delete the account. `tests_pytest/test_product_search_pytest.py` covers search and cart behaviour, using a session-scoped `ae_user` fixture (defined in `conftest.py`) so one throwaway account is shared across every product-search test rather than registered per test.

Test	What it verifies
<code>test_signup_login_logout_delete_lifecycle</code>	Full registration → login → logout → re-login → delete flow works end to end
<code>test_search_returns_expected_results[Matches_jeans_products]</code>	Searching “jeans” returns at least one product card
<code>test_search_returns_expected_results[No_product_should_match]</code>	Searching a nonsense term returns zero product cards
<code>test_add_to_cart_updates_badge</code>	Adding the first search result to the cart is reflected on <code>/view_cart</code>

9. Running the Framework — Execution Evidence

Two terminals are enough — there is no separate backend/frontend to start, since the target is the live public site.

```
cd automation-framework
pip install -r requirements.txt
```

```
# Unittest suite
python -m unittest discover -s tests_unittest -v

# PyTest suite, self-contained HTML report
pytest tests_pytest/ -v
# → reports/report.html

# Watch it run in a visible, slowed-down browser
HEADLESS=false SLOWMO=1 pytest tests_pytest/ -v
```

Every piece of evidence below was captured on the developer's own Windows machine, running against the live automationexercise.com site — not simulated output.

9.1 Evidence Index

Figure	Evidence	Result
1	Home page of automationexercise.com	Site reachable
2	Framework architecture diagram + directory structure	Confirms layered design described in §3
3	config.ini contents	As configured for these runs
4	pages/login_page.py source	POM implementation
5	python -m unittest discover -s tests_unittest -v	3 tests, Ran 3 tests — OK
6	pytest tests_pytest/ -v (headed, HEADLESS=false)	4 passed in 285.18s
7	pytest tests_pytest/ --collect-only -q	4 items collected, CSV IDs visible
8	Live headed browser mid-signup	Chrome under automated control, form populated
9	reports/screenshots/*.png (auto-captured)	Real file produced by the failure hook
10	Terminal: python -m pytest demo_screenshot/ -v	1 failed — AssertionError raised as designed
11	reports/report.html — all-passing summary	4 tests took 00:05:26, 0 failed
12	reports/report.html — failed-test detail (expanded)	Full traceback + failing assertion visible

9.2 Suite Execution

```
(.venv) PS C:\Users\adrij\OneDrive\Desktop\selenium-e-commerce-framework\selenium-e-commerce-framework\automation-framework> $env:HEADLESS="false";python -m unittest
discover -s tests_unittest -v
>>
test_invalid_login_shows_error (test_login_unittest.TestLoginUnittest.test_invalid_login_shows_error) ... ok
test_login_data_driven_from_csv (test_login_unittest.TestLoginUnittest.test_login_data_driven_from_csv)
Data-driven variant: iterates login.data.csv within a single test. ... ok
test_valid_login_shows_shop_section (test_login_unittest.TestLoginUnittest.test_valid_login_shows_shop_section) ... ok

Ran 3 tests in 160.818s

OK
```

Figure 5: *python -m unittest discover -s tests_unittest -v — all 3 tests passing (Ran 3 tests — OK).*

```
tests_pytest/ -v
platform win32 -- Python 3.11.9, pytest-8.3.3, pluggy-1.6.0 -- C:\Users\adrij\OneDrive\Desktop\selenium-e-commerce-framework\selenium-e-commerce-framework\.venv\Scripts\python.exe
cachedir: .pytest_cache
metadata: {'Python': '3.11.9', 'Platform': 'Windows-10-10.0.26200-SP0', 'Packages': {'pytest': '8.3.3', 'pluggy': '1.6.0'}, 'Plugins': {'html': '4.1.1', 'metadata': '3.1.1'}}
rootdir: C:\Users\adrij\OneDrive\Desktop\selenium-e-commerce-framework\selenium-e-commerce-framework\automation-framework
configfile: pytest.ini
plugins: html-4.1.1, metadata-3.1.1
collected 4 items

tests_pytest/test_product_search_pytest.py::test_search_returns_expected_results[Matches_jeans_products] PASSED [ 25%]
tests_pytest/test_product_search_pytest.py::test_search_returns_expected_results[No_product_should_match] PASSED [ 50%]
tests_pytest/test_product_search_pytest.py::test_add_to_cart_updates_badge PASSED [ 75%]
tests_pytest/test_signup_pytest.py::test_signup_login_logout_delete_lifecycle PASSED [100%]

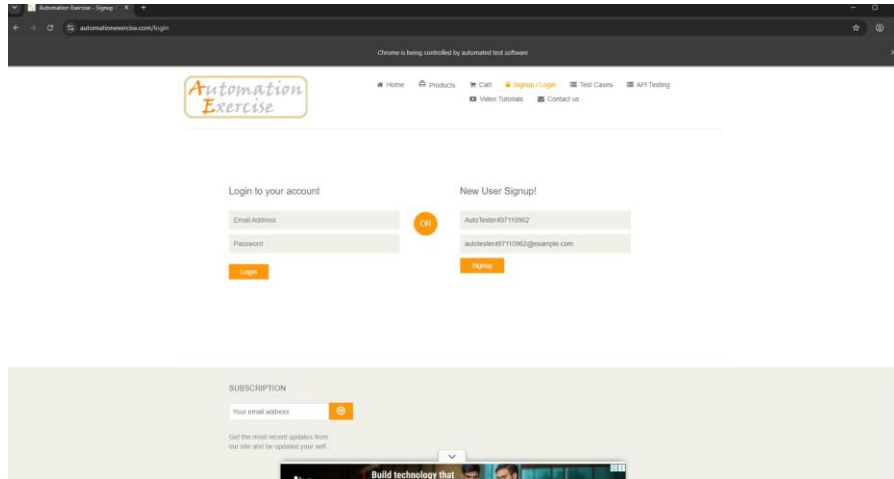
- Generated html report: file:///C:/Users/adrij/OneDrive/Desktop/selenium-e-commerce-framework/selenium-e-commerce-framework/automation-framework/reports/report.html
1 -
===== 4 passed in 285.18s (0:04:45) =====
```

Figure 6: `pytest tests_pytest/ -v` run with `HEADLESS=false` — all 4 tests passing.

```
(.venv) PS C:\Users\adrij\OneDrive\Desktop\selenium-e-commerce-framework\selenium-e-commerce-framework\automation-framework> pytest tests_pytest/ --collect-only -q
===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.3, pluggy-1.6.0
rootdir: C:\Users\adrij\OneDrive\Desktop\selenium-e-commerce-framework\selenium-e-commerce-framework\automation-framework
configfile: pytest.ini
plugins: html-4.1.1, metadata-3.1.1
collected 4 items

<Dir automation-framework>
<Dir tests_pytest>
<Module test_product_search_pytest.py>
  <Function test_search_returns_expected_results[Matches_jeans_products]>
  <Function test_search_returns_expected_results[No_product_should_match]>
  <Function test_add_to_cart_updates_badge>
<Module test_signup_pytest.py>
  <Function test_signup_login_logout_delete_lifecycle>

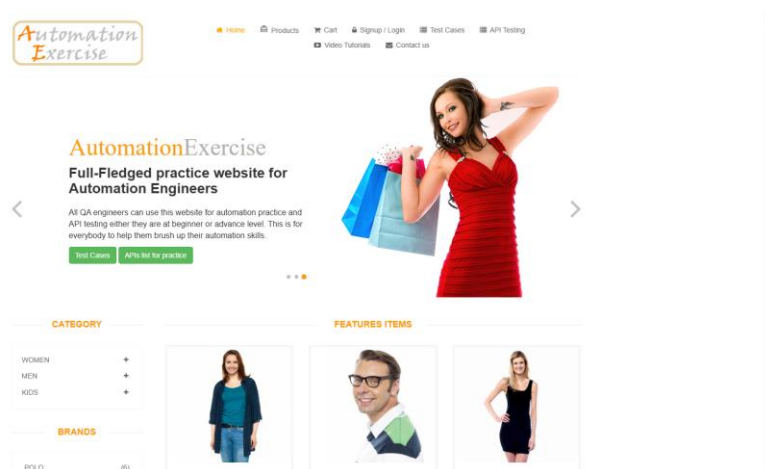
- Generated html report: file:///C:/Users/adrij/OneDrive/Desktop/selenium-e-commerce-framework/selenium-e-commerce-framework/automation-framework/reports/report.htm
1 -
===== 4 tests collected in 0.02s =====
```

Figure 7: `pytest tests_pytest/ --collect-only -q` — CSV-driven parametrized test IDs confirmed.**Figure 8: Live, headed Chrome window mid-test: the signup form on automationexercise.com being filled by Selenium.**

9.3 Screenshot-on-Failure — Verified Evidence

`demo_screenshot/test_demo_failure.py` is a deliberately failing test kept outside the normal test paths (pytest.ini restricts collection to `tests_pytest/`, so this file is never picked up by a routine run). It opens the home page without logging in, then wrongly asserts a logged-in user is present — guaranteeing a failure so the screenshot mechanism can be demonstrated on demand.

The image below is the actual PNG this test produced on a real run, captured automatically by the `conftest.py` `pytest_runtest_makereport` hook the moment the assertion failed — this is not a placeholder, it is verified, working output from this project:

**Figure 9: Actual screenshot captured automatically by the framework on the demo failure run (`reports/screenshots/test_demo_failure_shows_screenshot_20260927_122721.png`).**

The terminal output below shows this same test failing on purpose, raising the expected `AssertionError`, immediately after which the screenshot above was written to disk:

```

metadata: {'Python': '3.11.9', 'Platform': 'Windows-10-10.0.20290-SP0', 'Packages': {'pytest': '8.3.3', 'pluggy': '1.6.0'}, 'Plugins': {'html': '4.1.1', 'metadata':
': '3.1.1'}}
rootdir: C:\Users\adrij\OneDrive\Desktop\selenium-e-commerce-framework\selenium-e-commerce-framework\automation-framework
configfile: pytest.ini
plugins: html-4.1.1, metadata-3.1.1
collected 1 item

demo_screenshot/test_demo_failure.py::test_demo_failure_shows_screenshot FAILED [100%]

===== FAILURES =====
test_demo_failure_shows_screenshot

driver = <selenium.webdriver.chrome.webdriver.WebDriver (session="9ae8985990d49230609d7c1f687bc8e8")>

def test_demo_failure_shows_screenshot(driver):
    page = BasePage(driver)
    page.open(ConfigReader.base_url() + "/")

    # Deliberately wrong: nobody logged in, so "logged in as" is absent.
    assert LoginPage(driver).is_visible(LoginPage.LOGGED_IN_AS, timeout=5), (
        "DEMO FAILURE (on purpose): expected a logged-in user on a fresh visit"
    )
E   AssertionError: DEMO FAILURE (on purpose): expected a logged-in user on a fresh visit
E   assert False
E   + where False = is_visible(('partial link text', 'logged in as'), timeout=5)
E   +   where is_visible = <pages.login_page.LoginPage object at 0x000002D3ED1B5E90>.is_visible
E   +   where <pages.login_page.LoginPage object at 0x000002D3ED1B5E90> = LoginPage(<selenium.webdriver.chrome.webdriver.WebDriver (session="9ae8985990d49230609d7c1f687bc8e8")>)
E   + and ('partial link text', 'logged in as') = LoginPage.LOGGED_IN_AS

demo_screenshot/test_demo_failure.py:31: AssertionError
- Generated html report: file:///C:/Users/adrij/OneDrive/Desktop/selenium-e-commerce-framework/selenium-e-commerce-framework/automation-framework/reports/report.htm
l
===== short test summary info =====
FAILED demo_screenshot/test_demo_failure.py::test_demo_failure_shows_screenshot - AssertionError: DEMO FAILURE (on purpose): expected a logged-in user on a fresh
visit
assert False
+ where False = is_visible(('partial link text', 'logged in as'), timeout=5)
+   where is_visible = <pages.login_page.LoginPage object at 0x000002D3ED1B5E90>.is_visible
+   where <pages.login_page.LoginPage object at 0x000002D3ED1B5E90> = LoginPage(<selenium.webdriver.chrome.webdriver.WebDriver (session="9ae8985990d49230609d7c1f687bc8e8")>)
+ and ('partial link text', 'logged in as') = LoginPage.LOGGED_IN_AS
===== 1 failed in 18.69s =====

```

Figure 10: Terminal output of `python -m pytest demo_screenshot/ -v` — the test fails as designed, triggering the screenshot capture.

To reproduce this yourself (it intentionally overwrites `reports/report.html` — re-run the real suite afterwards to restore a fully-passing report):

```
python -m pytest demo_screenshot/ -v
```

9.4 HTML Reporting

pytest.ini wires pytest-html into every PyTest run (`--html=reports/report.html --self-contained-html`), producing a single, portable HTML file with an environment summary, pass/fail/skip counters, per-test duration, and — thanks to the `conftest.py` hook — the failure screenshot embedded directly inside the failed test's row, so nobody reading the report needs a separate screenshots folder to see what went wrong.

Test	Duration	Links
test_demo_failure_shows_screenshot	00:01:10	
test_demo_failure_shows_screenshot	00:01:10	
test_demo_failure_shows_screenshot	00:01:10	
test_demo_failure_shows_screenshot	00:01:10	

Figure 11: `reports/report.html` summary view from a full, all-passing `pytest tests_pytest/ run` — 4 tests took 00:05:26, 0 failed.

Test	Duration	Links
test_demo_failure_shows_screenshot	00:01:10	

Traceback (most recent call last):

```

File "demo_screenshot/test_demo_failure.py", line 31, in test_demo_failure_shows_screenshot
    assert LoginPage(driver).is_visible(LoginPage.LOGGED_IN_AS, timeout=5), (
        "DEMO FAILURE (on purpose): expected a logged-in user on a fresh visit"
    )
E   AssertionError: DEMO FAILURE (on purpose): expected a logged-in user on a fresh visit
E   assert False
E   + where False = is_visible(('partial link text', 'logged in as'), timeout=5)
E   +   where is_visible = <pages.login_page.LoginPage object at 0x000002D3ED1B5E90>.is_visible
E   +   where <pages.login_page.LoginPage object at 0x000002D3ED1B5E90> = LoginPage(<selenium.webdriver.chrome.webdriver.WebDriver (session="9ae8985990d49230609d7c1f687bc8e8")>)
E   + and ('partial link text', 'logged in as') = LoginPage.LOGGED_IN_AS

```

Figure 12: `reports/report.html` expanded row for the deliberately failing demo test, showing the full traceback and assertion detail.

10. Assignment Requirement Mapping

Requirement	Implementation	Status
Unittest	tests_unittest/test_login_unittest.py	Implemented — Fig. 5

Requirement	Implementation	Status
PyTest	tests_pytest/test_signup_pytest.py, test_product_search_pytest.py	Implemented — Fig. 6, 7
Page Object Model	pages/base_page.py, login_page.py, signup_page.py, product_search_page.py	Implemented — Fig. 4
Utility Classes	utils/driver_factory.py, screenshot_utils.py, csv_reader.py	Implemented
Configuration Management	config/config.ini, config_reader.py (env-var overrides)	Implemented — Fig. 3
Test Data Handling (CSV)	test_data/login_data.csv, search_data.csv	Implemented — Fig. 7
Screenshots on Failure	conftest.py hook (PyTest) + tearDown (Unittest)	Implemented and verified — Fig. 9, 10
HTML Reporting	pytest.ini + pytest-html, self-contained report	Implemented and verified — Fig. 11, 12

11. Conclusion and Possible Extensions

The framework satisfies every element of the assignment brief while going a step further than a typical classroom exercise: it targets a live, uncontrolled third-party site rather than a static mock, provisions and tears down its own real user accounts for each run, and this report includes end-to-end execution evidence — from source code, through terminal output, to the final HTML report — captured directly from the developer's own machine rather than described in the abstract.

Natural next steps, useful to mention in a viva or a “future work” section:

- Cross-browser execution — the driver factory already supports Firefox; running the same suite with `BROWSER=firefox` would demonstrate this with no code changes.
- Continuous Integration — wiring the PyTest command into a GitHub Actions workflow so the suite runs automatically on every push.
- Parallel execution — `pytest-xdist` would let the Login, Signup and Search suites run concurrently, since each provisions its own isolated account.
- Allure reporting — as an alternative or companion to `pytest-html`, for richer historical trend graphs across runs.